

Online Ticket Booking System Design

Introduction

An Online Ticket Booking System enables users to **discover events, browse show details, reserve seats, complete payments, and receive digital tickets**. Unlike a traditional CRUD application, this system operates under strict scalability and consistency constraints. It must handle extremely high traffic volumes generated by millions of users while simultaneously ensuring that critical business operations, such as seat allocation, remain accurate and free from conflicts.

The most challenging aspect of designing such a platform is that different subsystems have fundamentally different priorities. **Event discovery demands high availability and low latency because users expect search results to appear instantly, even during periods of heavy traffic**. In contrast, **the booking subsystem prioritizes correctness over availability because allocating the same seat to multiple users is unacceptable**. Consequently, different parts of the architecture make different trade-offs based on their business requirements.

Functional Requirements

The platform should allow users to search for events using attributes such as event title, location, and date. After identifying an event of interest, users should be able to view detailed information including venue details, event descriptions, show timings, pricing information, and the corresponding seat map. Users must then be able to select seats and initiate the booking process. The system should temporarily reserve the selected seats to prevent concurrent users from proceeding with the same inventory during checkout. Once payment is successfully completed, the booking should be permanently confirmed and digital tickets should be issued. Users should also be able to review their booking history and receive notifications through email or SMS regarding the status of their reservations.

Non-Functional Requirements

The system is expected to support approximately **one hundred million daily active users** while accommodating sudden spikes generated by blockbuster movie releases, concerts, and sporting events. The workload is predominantly read-oriented, with approximately **ninety percent of requests involving browsing and searching activities and only ten percent involving booking transactions**.

Search operations should maintain response times below two hundred milliseconds to provide a seamless browsing experience. However, booking workflows prioritize correctness over speed. Slightly increased latency during seat confirmation is acceptable if it guarantees that duplicate seat allocation never occurs.

CAP Theorem Considerations

The architecture deliberately adopts different CAP theorem trade-offs for different subsystems.

The event search subsystem prioritizes Availability and Partition Tolerance. During periods of network partitions or failures, users should still be able to search and browse events without interruption. A minor delay before newly updated information appears in search results is considered acceptable. Therefore, the search subsystem embraces eventual consistency and follows an AP-oriented approach.

In contrast, the seat booking subsystem prioritizes Consistency and Partition Tolerance. The same seat for the same showtime must never be sold to multiple users. If a partition occurs and the system cannot guarantee correctness, it is preferable to temporarily reject booking requests rather than risk inconsistent allocations. Therefore, seat booking follows a CP-oriented approach where consistency takes precedence over availability.

High-Level Architecture

The system follows a **microservices** architecture. All incoming requests from end users and administrators first pass through a load balancer, which distributes traffic across multiple application instances. Requests are then forwarded to an API Gateway that acts as the unified entry point into the system.

The API Gateway is responsible for **authentication, authorization, request routing, rate limiting, monitoring, and request tracing**. Based on the nature of the request, traffic is routed to the Search Service, Admin Service, or Booking and Payment Service.

The Search Service handles event discovery. The Admin Service manages event inventory, venue configurations, pricing, and scheduling. The Booking and Payment Service coordinates seat reservation, payment processing, booking confirmation, and notification generation.

Search Architecture and Eventual Consistency

Since the majority of traffic consists of browsing activities, the search subsystem is optimized for high availability and fast reads. **Elasticsearch** is selected as the search engine because it provides full-text search capabilities, filtering, relevance ranking, and horizontal scalability.

Operational event information is maintained within Cassandra. However, Cassandra is not optimized for search workloads. To bridge this gap, a Change Data Capture pipeline continuously transfers updates from Cassandra into Elasticsearch. Whenever administrators modify events, create showtimes, adjust pricing, or update venue information, these changes are propagated through the CDC pipeline. Elasticsearch then refreshes its indexes and eventually reflects the latest state.

Because the search subsystem prioritizes availability, slight synchronization delays are acceptable. This design enables users to continue browsing events even during periods of infrastructure degradation.

Seat Reservation Workflow

The booking workflow represents the most critical component of the system because it directly affects customer trust.

When a user selects seats for a particular showtime, the Booking Service first attempts to create a temporary reservation using **Redis**. Redis is chosen because of its extremely low latency and support for atomic operations. Each selected seat is associated with a unique Redis key containing information such as the show identifier, seat identifier, user identifier, and reservation expiration timestamp.

If the reservation operation succeeds, the seat enters an intermediate state known as "under booking." This state indicates that another user is currently attempting to complete the checkout process. ***Other users attempting to reserve the same seat will be prevented from proceeding because Redis guarantees atomic access to the reservation key.***

Whenever Redis successfully creates a temporary reservation, ***a CDC process propagates this reservation information into Cassandra.*** Cassandra therefore becomes aware not only of permanently booked seats but also of seats currently undergoing checkout. These updates subsequently flow from Cassandra into Elasticsearch through the existing CDC pipeline.

As a result, users viewing seat maps through Elasticsearch can immediately observe that certain seats are currently unavailable because they are under reservation. This prevents users from repeatedly attempting to select seats that are already being processed by another customer.

Reservation State Propagation

The propagation of seat state follows a sequential flow.

When Redis records a temporary hold, the reservation state is transmitted into Cassandra. Cassandra maintains a durable representation of the current seat inventory, including seats under active reservation. Through Change Data Capture, Cassandra then updates Elasticsearch. Consequently, the search layer remains aware of temporary booking activity and exposes this information to end users.

This architecture significantly improves user experience because users no longer encounter seats that appear available but subsequently fail during checkout.

Payment Processing and Booking Confirmation

Once seats have been successfully reserved, the Booking Service initiates payment processing through the integrated payment gateway.

If payment succeeds, the Booking Service executes a strongly consistent transactional workflow using PostgreSQL. The booking record is created, payment information is persisted, ticket information is generated, and the seat status is permanently updated within the transactional database. Following successful transaction completion, ***the corresponding Redis reservation keys are removed because the temporary reservation has now transitioned into a confirmed booking.***

This sequence ensures that Redis remains responsible only for short-lived reservation management while PostgreSQL acts as the source of truth for completed bookings and financial records.

If payment fails, the reservation keys are immediately removed from Redis. The associated seat status eventually propagates through Cassandra and Elasticsearch, allowing other users to reserve those seats.

Handling Reservation Expiration and Edge Cases

One of the most important edge cases arises when the Redis reservation expires at approximately the same moment that payment succeeds.

Consider a scenario in which a reservation is configured to expire after fifteen minutes. A user initiates payment during the final few seconds of this reservation period. If Redis automatically removes the reservation before the Booking Service completes payment confirmation, ***another user could potentially attempt to reserve the same seat.***

To prevent this situation, robust communication between the payment subsystem and booking subsystem is essential.

Before finalizing a booking, the Booking Service must revalidate the reservation state and extend the reservation if payment processing is already in progress. Alternatively, the payment gateway callback should include an idempotent confirmation mechanism that verifies whether the reservation had previously existed and whether the booking transaction has already been initiated.

The booking confirmation process should therefore operate within a transactional boundary that guarantees exactly-once execution. If payment succeeds before expiration, the reservation must be promoted into a permanent booking even if the Redis expiration event has been triggered. Conversely, if no successful payment confirmation exists, the reservation expiration should safely release the seat back into inventory.

This careful coordination between Redis, PostgreSQL, and the payment subsystem prevents race conditions and ensures that users are neither charged without receiving tickets nor issued duplicate bookings.

Strong Consistency for Seat Allocation

Preventing duplicate bookings requires multiple layers of protection.

Redis provides the first line of defense by serializing concurrent reservation attempts. PostgreSQL provides the second layer through ACID transactions that guarantee atomic execution of booking operations. Finally, database-level unique constraints on the combination of show identifier and seat identifier ensure that duplicate records cannot exist even under extreme concurrency conditions.

Therefore, even if unexpected failures occur, the architecture mathematically guarantees that the same seat cannot be permanently assigned to multiple users.

Notifications and Booking History

Following successful confirmation, the Booking Service publishes booking events to downstream notification systems. Dedicated notification services generate confirmation emails, SMS messages, or push notifications containing ticket details and booking references.

Because notifications execute asynchronously, delays within external communication providers do not impact booking performance.

Users can subsequently access their booking history through dedicated APIs, enabling them to review upcoming events, previously attended events, and booking statuses.

Conclusion

The proposed Online Ticket Booking System demonstrates how different architectural choices can coexist within a single platform to satisfy conflicting business requirements. Event discovery prioritizes availability and responsiveness, adopting an AP-oriented model supported by Cassandra, CDC pipelines, and Elasticsearch. Seat allocation prioritizes correctness, adopting a CP-oriented model built upon Redis-based reservations, transactional PostgreSQL confirmations, and database-level uniqueness guarantees.

The introduction of reservation synchronization from Redis to Cassandra and subsequently to Elasticsearch ensures that end users receive an accurate representation of seats currently undergoing checkout. Furthermore, careful coordination between Redis expiration, payment callbacks, and transactional confirmation logic addresses critical edge cases involving last-second payments and reservation expiry.

By combining these techniques, the system achieves high scalability, low search latency, fault tolerance, and most importantly, absolute confidence that a single seat for a given showtime can never be sold to more than one customer.

